

Chap8_Main_Memory

8.1 background

The **base register** holds the smallest legal physical memory address; the **limit register** specifies the size of the range. For example, if the base register holds

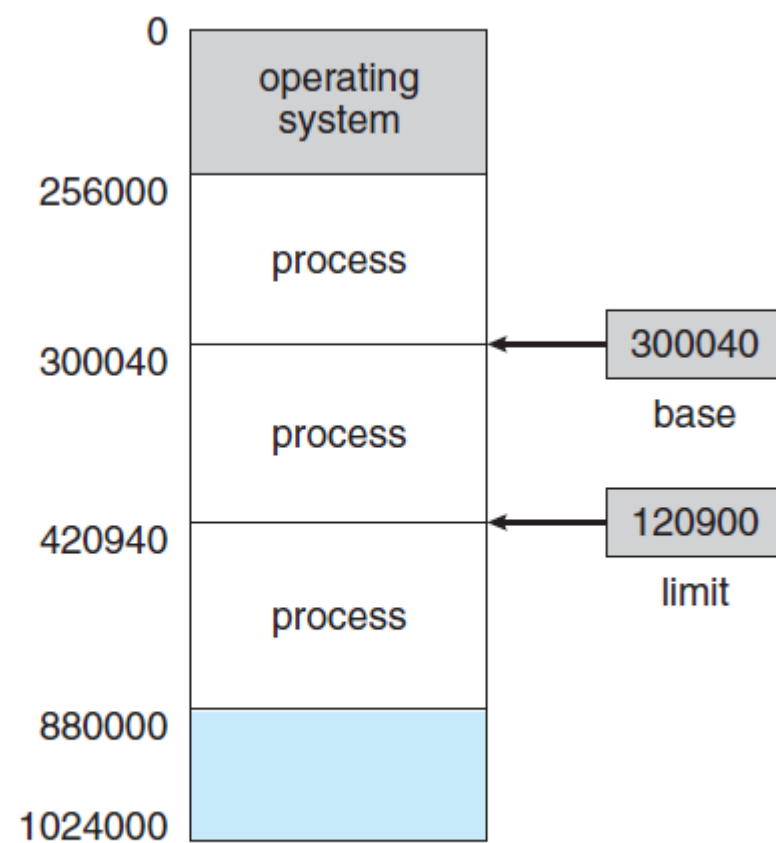


Figure 8.1 A base and a limit register define a logical address space.

the operating system or other users.

The base and limit registers can be loaded only by the operating system, which uses a special privileged instruction. Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.

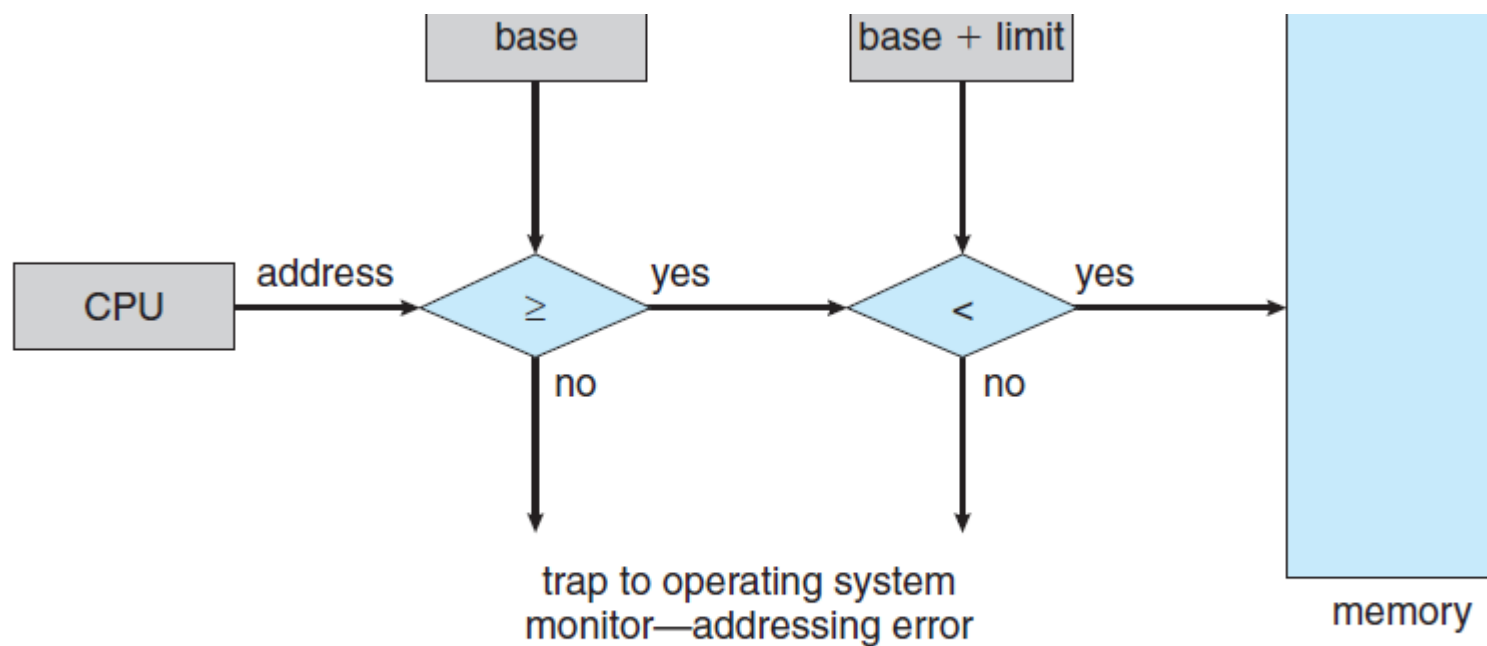


Figure 8.2 Hardware address protection with base and limit registers.

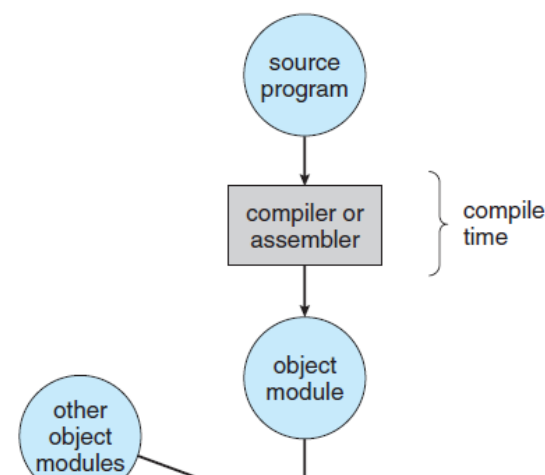
8.1.2 Address Binding

In most cases, a user program goes through several steps—some of which may be optional—before being executed (Figure 8.3). Addresses may be represented in different ways during these steps. Addresses in the source program are generally symbolic (such as the variable `count`). A compiler typically **binds** these symbolic addresses to relocatable addresses (such as “14 bytes from the beginning of this module”). The linkage editor or loader in turn binds the relocatable addresses to absolute addresses (such as 74014). Each binding is a mapping from one address space to another.

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

Classically, the binding of instructions and data to memory addresses can be done at any step along the way:

- **Compile time.** If you know at compile time where the process will reside in memory, then **absolute code** can be generated. For example, if you know that a user process will reside starting at location *R*, then the generated compiler code will start at that location and extend up from there. If, at some later time, the starting location changes, then it will be necessary to recompile this code. The MS-DOS .COM-format programs are bound at compile time.
- **Load time.** If it is not known at compile time where the process will reside in memory, then the compiler must generate **relocatable code**. In this case, final binding is delayed until load time. If the starting address changes, we need only reload the user code to incorporate this changed value.



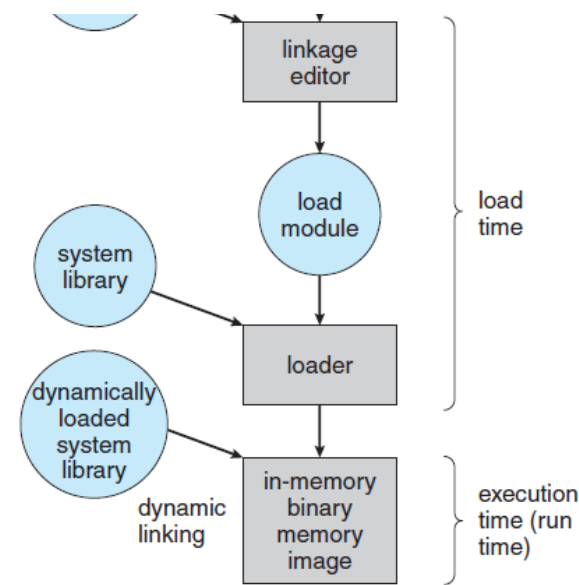


Figure 8.3 Multistep processing of a user program.

- **Execution time.** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time. Special hardware must be available for this scheme to work, as will be discussed in Section 8.1.3. Most general-purpose operating systems use this method.

8.1.3 Logical vs Physical Address space

- Logical address generated by CPU
- Physical address is the actual address in the main memory(RAM)
- Relocation register has the base address, and is contained inside the MMU(Memory Management Unit)

8.1.4 Dynamic Loading

- With dynamic loading, a routine is not loaded until it is called

8.1.5 Dynamic Linking and Shared Libraries (Search for this topic on net, it's not clear)

[Linking vs Loading](#)

- Dynamic linked libraries are **system libraries** that are linked to user program when the programs are run.
- **Static linking**, in which system libraries are treated like any other object module and are combined by the loader into the binary program image
- **Dynamic Linking** is similar to dynamic loading, here **though linking rather than loading is postponed until execution time.**
- This feature can be extended to library updates (such as bug fixes).
 - A library may be replaced by a new version, and all programs that reference the library will automatically use the new version.
 - Without dynamic linking, all such programs would need to be relinked to gain access to the new library.
- Unlike dynamic loading, dynamic linking and shared libraries generally require help from the operating system.

Loading and Linking are further categorized into 2 types:

STATIC	DYNAMIC
Loading the entire program into the main memory before start of the program execution is called as static loading.	Loading the program into the main memory on demand is called as dynamic loading.
Inefficient utilization of memory because whether it is required or not required entire program is brought into the main memory.	Efficient utilization of memory.

Program execution will be faster.

Program execution will be slower.

Statically linked program takes constant load time every time it is loaded into the memory for execution.

Dynamic linking is performed at run time by the operating system.

If the static loading is used then accordingly static linking is applied.

If the dynamic loading is used then accordingly dynamic linking is applied.

Static linking is performed by programs called linkers as the last step in compiling a program. Linkers are also called link editors.

In dynamic linking this is not the case and individual shared modules can be updated and recompiled. This is one of the greatest advantages dynamic linking offers.

In static linking if any of the external programs has changed then they have to be recompiled and re-linked again else the changes won't reflect in existing executable file.

In dynamic linking load time might be reduced if the shared library code is already present in memory.

• 8.2 Swapping

- Used with a backing store, to store idle processes to increase the extent of multiprogramming
- The context switch time is fairly high, and major part of it is transfer time which is proportional to the amount of memory swapped.
- The process should be **in idle state when it is swapped**.
- Swapping is normally disabled but will start if amount of free memory falls below a threshold amount.
- Swapping in mobiles is disabled because of space constraints. (they use flash memory which can only tolerate a limited number of write operations.)
- Instead of swapping, android saves the app state to flash memory to restart it quickly. **But both iOS and Android support paging**
-

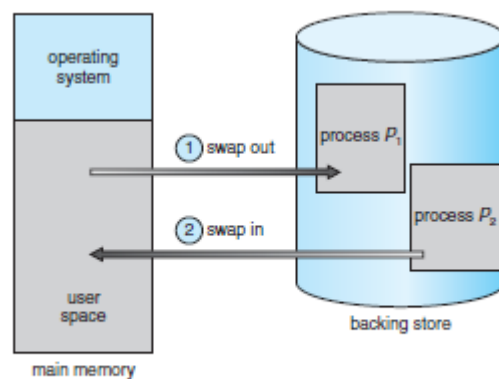


Figure 8.5 Swapping of two processes using a disk as a backing store.

- enough to accommodate copies of all memory images for all users, and it must provide direct access to these memory images. The system maintains a ready queue consisting of all processes whose memory images are on the backing store or in memory and are ready to run. Whenever the CPU scheduler decides to execute a process, it calls the dispatcher. The dispatcher checks to see whether the next process in the queue is in memory. If it is not, and if there is no free

the next process in the queue is in memory. If it is not, and if there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the desired process. It then reloads registers and transfers control to the selected process.

The context-switch time in such a swapping system is fairly high. To get an

- Swaping is constrained by factors such as
 - Transfer time is proportional to the amount of data to be swapped
 - The process to be swapped needs to be **completely idle**.
 - Suppose a process is waiting for an I/O operation is pending, and we want to swap this process.
 - However, if the I/O is asynchronously accessing the user memory for I/O buffers, then the process cannot be swapped.
 - Assume that the I/O operation is queued because the device is busy.
 - If we were to swap out process P1 and swap in process P2, the I/O operation might then attempt to use memory that now belongs to process P2.
 - Solutions to this:
 - Never swap a pending I/O operation. Exactly what **the 'SYNC' option does in FSTAB file**
 - Execute I/O operations only in Operating systems buffers.
 - Transfer between Operating sytems buffer and process memory happens only when the process is swapped in. Called **DOUBLE BUFFERING. which adds overhead**
 - Standard swapping is not used but a modified version is used in UNIX Windows,
 - In one variation, it is disabled until the amount of free memory falls below a threshold. Halted when free memory increases.
 - Work alongside **VIRTUAL MEMORY**

• 8.3 Contiguous Memory allocation

◦

8.3.1 Memory Protection

Before discussing memory allocation further, we must discuss the issue of memory protection. We can prevent a process from accessing memory it does not own by combining two ideas previously discussed. If we have a system with a relocation register (Section 8.1.3), together with a limit register (Section 8.1.1), we accomplish our goal. The relocation register contains the value of the smallest physical address; the limit register contains the range of loaded

◦ 8.3.2 Memory Allocation

Now we are ready to turn to memory allocation. One of the simplest methods for allocating memory is to divide memory into several fixed-sized **partitions**. Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions. In this **multiple-partition method**, when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process. This method was originally used for the early operating systems (called **batch systems**) in which the user

- In the **variable-partition** scheme, the operating system keeps a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory, a **hole**. Eventually, as you will see, memory contains a cat of holes of various sizes

- - **First fit.** Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.
 - **Best fit.** Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produces the smallest leftover hole.
 - **Worst fit.** Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

Simulations have shown that both first fit and best fit are better than worst fit in terms of decreasing time and storage utilization. Neither first fit nor best fit is clearly better than the other in terms of storage utilization, but first fit is

generally faster.

8.3.3 Fragmentation

Both the first-fit and best-fit strategies for memory allocation suffer from **external fragmentation**. As processes are loaded and removed from memory, the free memory space is broken into little pieces. External fragmentation exists when there is enough total memory space to satisfy a request but the available spaces are not contiguous; storage is fragmented into a large number of small holes. This fragmentation problem can be severe. In the worst case, we could have a block of free (or wasted) memory between every two processes. If all these small pieces of memory were in one big free block instead, we might be able to run several more processes.

Whether we are using the first-fit or best-fit strategy can affect the amount of fragmentation. (First fit is better for some systems, whereas best fit is better for others.) Another factor is which end of a free block is allocated. (Which is the leftover piece—the one on the top or the one on the bottom?) No matter which algorithm is used, however, external fragmentation will be a problem.

Depending on the total amount of memory storage and the average process size, external fragmentation may be a minor or a major problem. Statistical analysis of first fit, for instance, reveals that, even with some optimization, given N allocated blocks, another $0.5 N$ blocks will be lost to fragmentation. That is, one-third of memory may be unusable! This property is known as the **50-percent rule**.

Memory fragmentation can be internal as well as external. Consider a multiple-partition allocation scheme with a hole of 18,464 bytes. Suppose that the next process requests 18,462 bytes. If we allocate exactly the requested block, we are left with a hole of 2 bytes. The overhead to keep track of this hole will be substantially larger than the hole itself. The general approach to avoiding this problem is to break the physical memory into fixed-sized blocks and allocate memory in units based on block size. With this approach, the memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is **internal fragmentation**—unused memory that is internal to a partition.

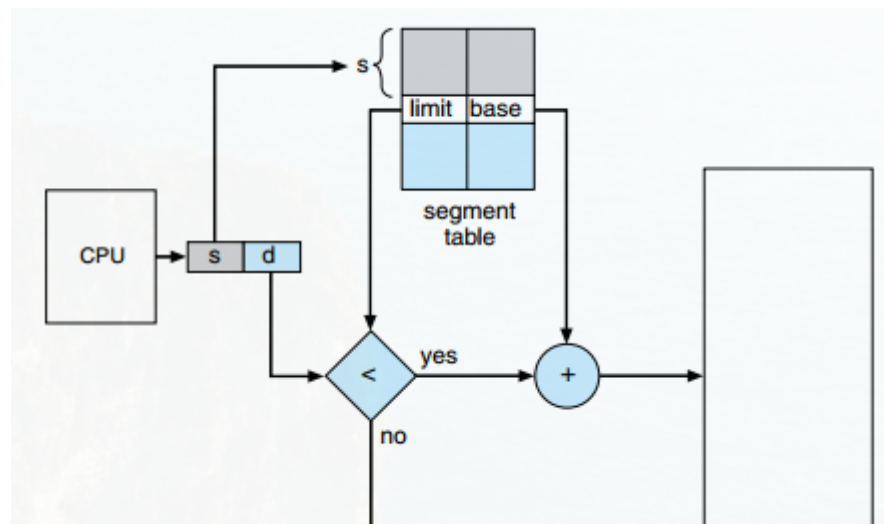
One solution to the problem of external fragmentation is **compaction**. The goal is to shuffle the memory contents so as to place all free memory together in one large block. Compaction is not always possible, however. If relocation is static and is done at assembly or load time, compaction cannot be done. It is possible only if relocation is dynamic and is done at execution time. If addresses are relocated dynamically, relocation requires only moving the program and data and then changing the base register to reflect the new base address. When compaction is possible, we must determine its cost. The simplest compaction algorithm is to move all processes toward one end of memory; all holes move in the other direction, producing one large hole of available memory. This scheme can be expensive.

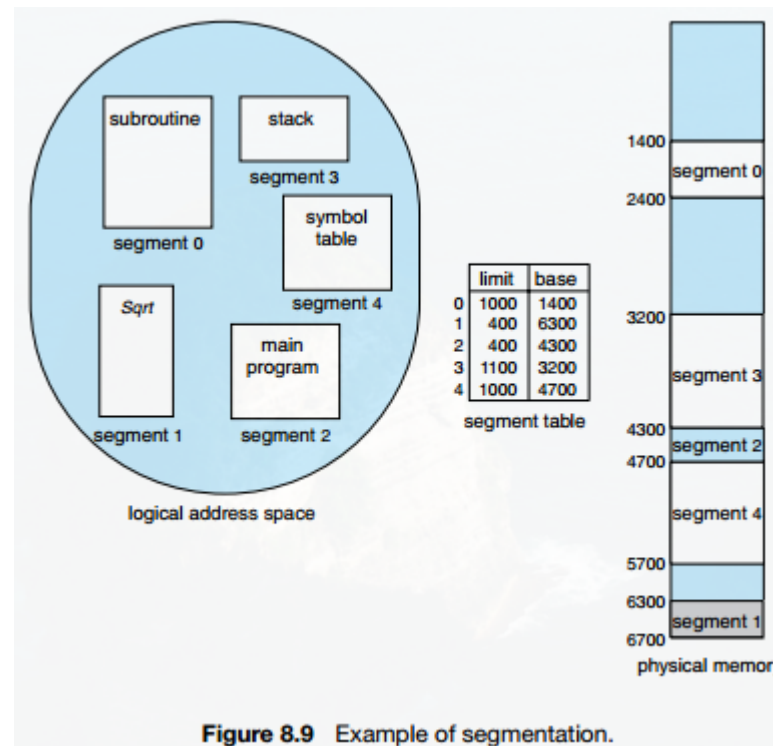
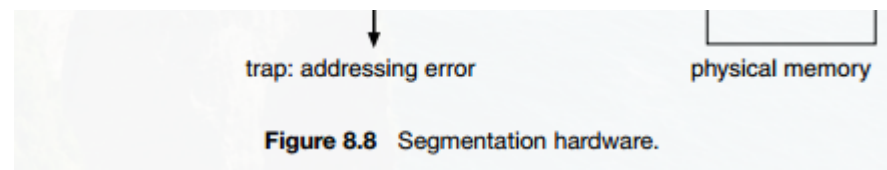
Another possible solution to the external-fragmentation problem is to permit the logical address space of the processes to be noncontiguous, thus allowing a process to be allocated physical memory wherever such memory is available. Two complementary techniques achieve this solution: **segmentation** (Section 8.4) and **paging** (Section 8.5). These techniques can also be combined.

Fragmentation is a general problem in computing that can occur wherever we must manage blocks of data. We discuss the topic further in the storage management chapters (Chapters 10 through and 12).

8.4 Segmentation

- Allows processes to have non-contiguous memory
- Supports the programmer's view of memory, a programmer might visualise codes in different segments, one for main method, one for binary tree search method
- Segments are two dimensional address, having the offset and segment number.
- Implemented using hardware called segment table, each entry of which has a segment base and a segment limit.





- This helps avoid external fragmentation of main memory

• 8.5 Paging

- This is another memory management scheme to solve the external fragmentation problem by using **fixed sized pages**. Although we have **internal fragmentation**.
- Paging also **avoids the need for compaction in the backing store**
 - In segmentation, the segment size is not fixed, so when swapping the segment we need to find the space on the backing store enough for it, and thus compaction would not always be possible.
- Page size is typically of the size in powers of 2.
- **Basic method:**
 - Break **physical memory** into fixed sized blocks called **frames** and break **logical memory** into blocks of same size called **pages**.
 - Backing store also, the same size of blocks.
 - The separation of frames and pages allows us **to have a logical address of 64 bit even though the system has less than 2^{64} bytes of memory**. This is why it is used in **VIRTUAL MEMORY ADDRESSING**.
 - Hardware



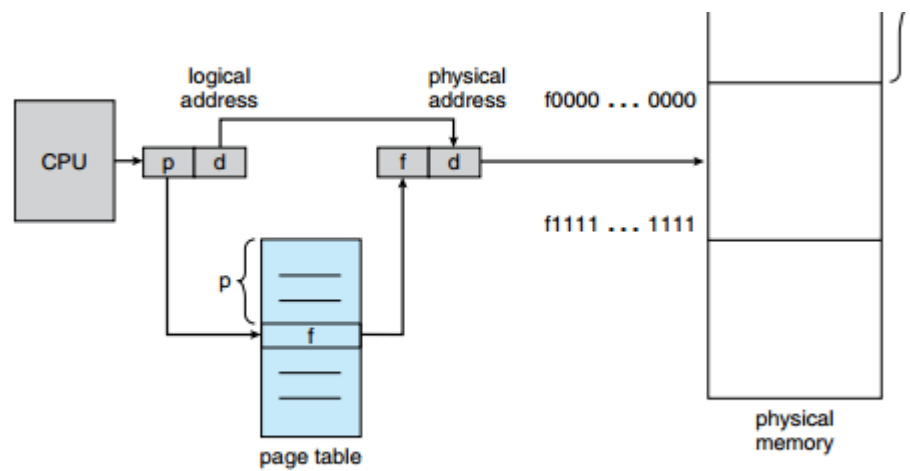


Figure 8.10 Paging hardware.

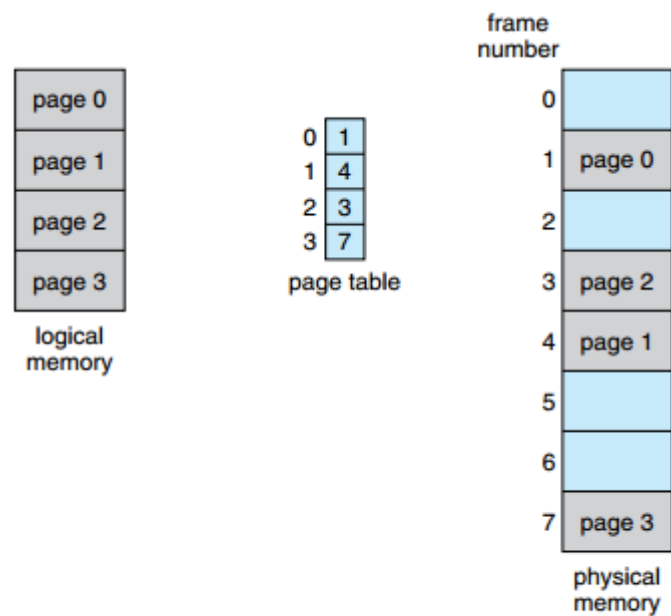
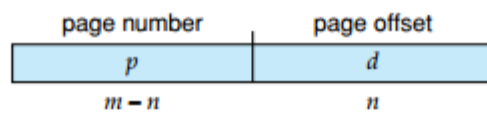
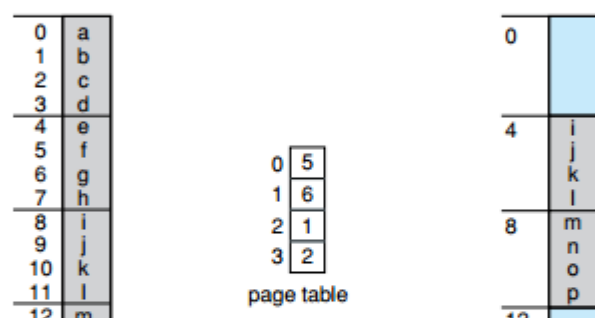


Figure 8.11 Paging model of logical and physical memory.



where p is an index into the page table and d is the displacement within the page.

As a concrete (although minuscule) example, consider the memory in Figure 8.12. Here, in the logical address, $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the programmer's view of memory can be mapped into physical memory. Logical address 0 is page 0, offset 0. Indexing into the page table, we find that page 0



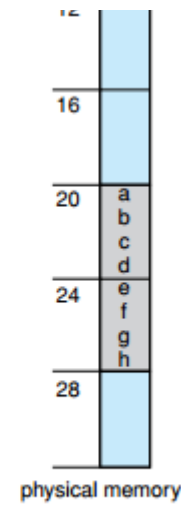
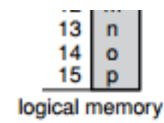
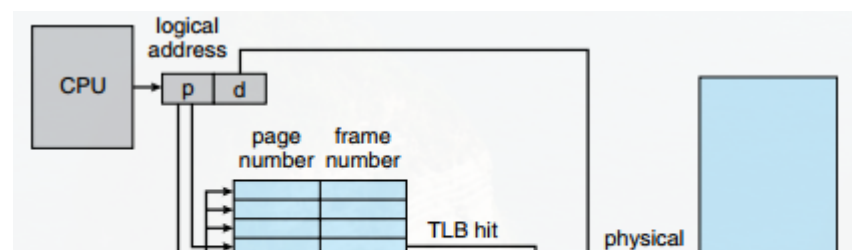


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

- There is also a **frame table**, which keeps the list of frames whether they are free or being occupied.
- Paging **also increases context-switch time**
- **Hardware support**
 - In simplest case, set of fast registers which is good for page tables which are small.
 - But when page tables get bigger, these become **infeasible**.
 - Rather the page table is kept in **main memory**, and can be accessed by a **single register**, called **PTBR(Page Table Base Register)**.
 - This reduces context-switch time.
 - But this method requires **2-fold memory access** one for page table entry and one for the byte. (Intolerable under most circumstances)
 - So, we need **Translation Look-aside Buffers TLBs**(Hardware which is like a fast memory).
 - It **compares all the entries simultaneously, with the pages' KEY**, so it is faster than walking through a whole page table
 - But smaller in size.
 - Only a given pages' address reside in TLB, not all
 - Consists of two parts:
 - key
 - value
 - It is **executed as part of the instruction pipeline within the CPU**, so no penalty on performance.
 - If page number not found then it is a **TLB miss**, and the page table is then searched.
 - If TLB is already full, then **replacement algorithms** are used like **LRU, Round robin, etc.**
 - Some instructions are **wired down** means they cannot be removed from the TLB
 - **ASIDs (Address Space Identifiers)**
 - Some TLBs store this to **uniquely identify each process and provide protection to that process**.
 - Needs to be **flushed**



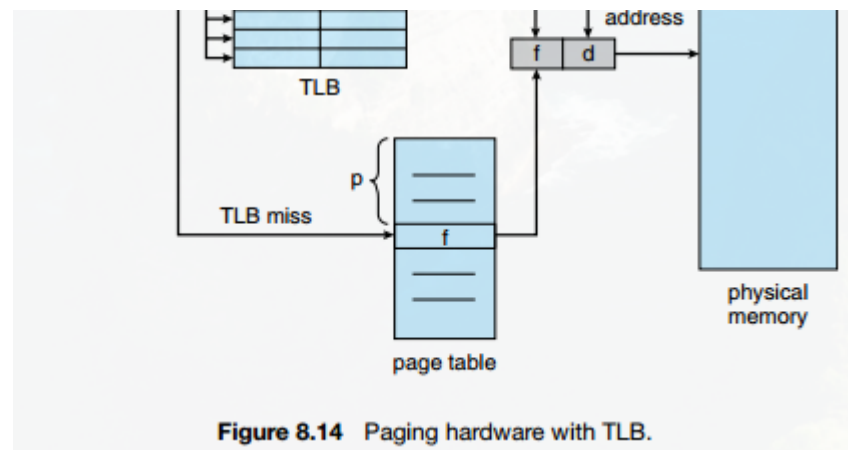
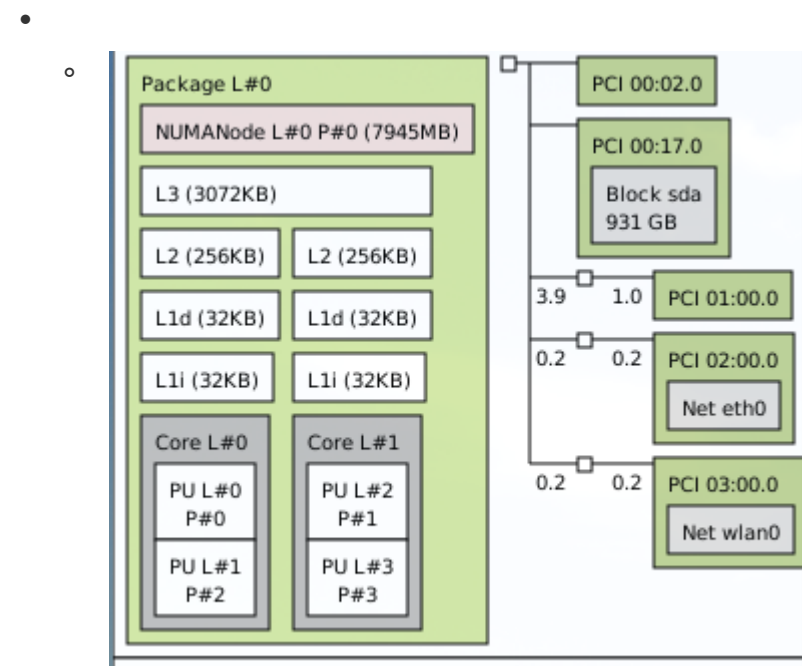


Figure 8.14 Paging hardware with TLB.

-
- HIT RATIO
 - (from 100 to 120 nanoseconds).
 - For a 99-percent hit ratio, which is much more realistic, we have

$$\begin{aligned} \text{effective access time} &= 0.99 \times 100 + 0.01 \times 200 \\ &= 101 \text{ nanoseconds} \end{aligned}$$



-
- Maybe different TLB for instruction and data
- Protection
 - Invalid and Valid bit in the page table.
 - One additional bit is generally attached to each entry in the page table: a **valid-invalid** bit. When this bit is set to *valid*, the associated page is in the process's logical address space and is thus a legal (or valid) page. When the bit is set to *invalid*, the page is not in the process's logical address space. Illegal addresses are trapped by use of the valid-invalid bit. The operating system sets this bit for each page to allow or disallow access to the page.

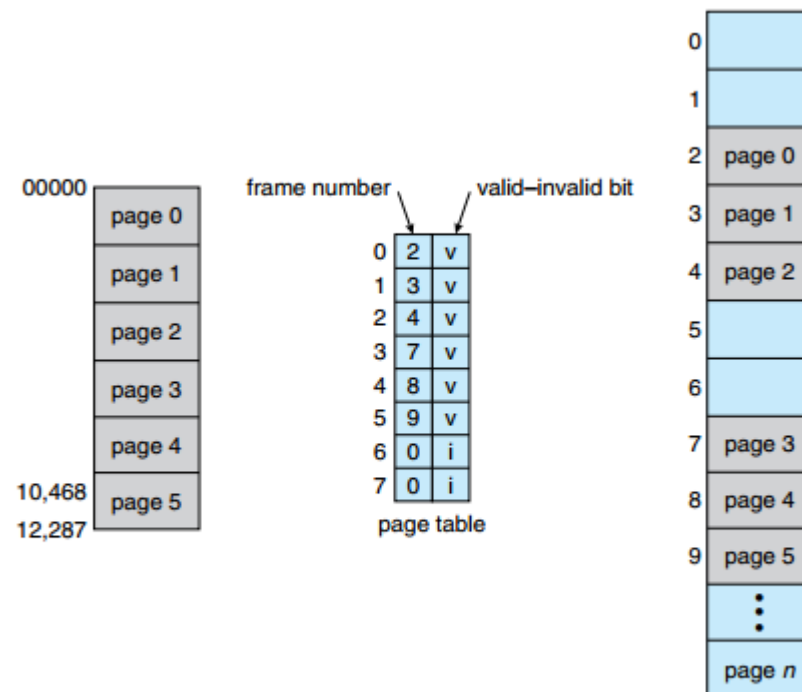


Figure 8.15 Valid (v) or invalid (i) bit in a page table.

- Problem of internal fragmentation still valid here, becoz of which valid addresses may be result in a trap.
 - Some systems provide hardware in teh form of **PTLR(Page table length register)**, to indicate the size of page table, so to avoid the above mentioned problem
- **Shared Pages**
- Pages can be shared among processes,
 - The process segregates the process into **data space (which is specific to all processes)** and **code space (which has code of program which can be shared)**

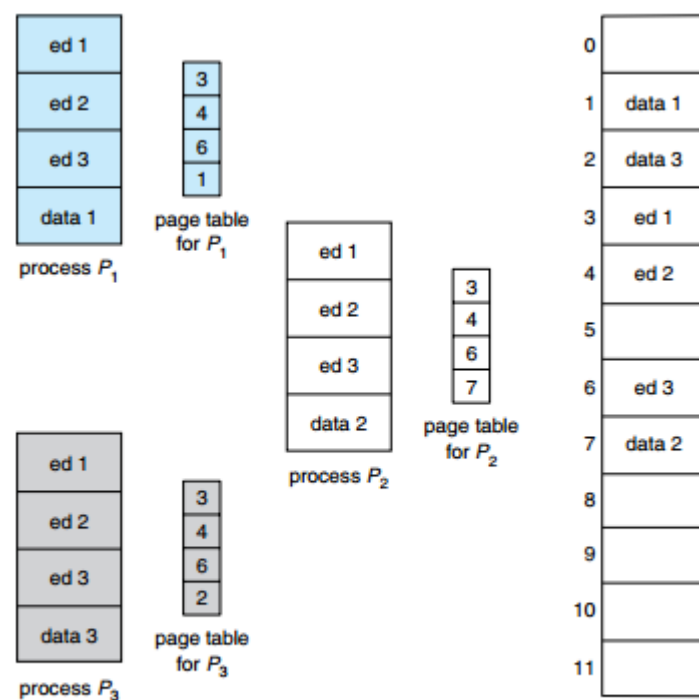


Figure 8.16 Sharing of code in a paging environment.

• 8.6 Page table structure

- The huge address space leads to huge page table, referencing to it for each virtual page number is inefficient.
- 32 bit systems are there but with the newer 64 bit systems, the focus on paging architecture exemplifies.
- Types:

- Hierarchical Paging
- Hashed Page table
- Inverted Page table
- Oracle SPARC Solaris

◦ Hierarchical Paging

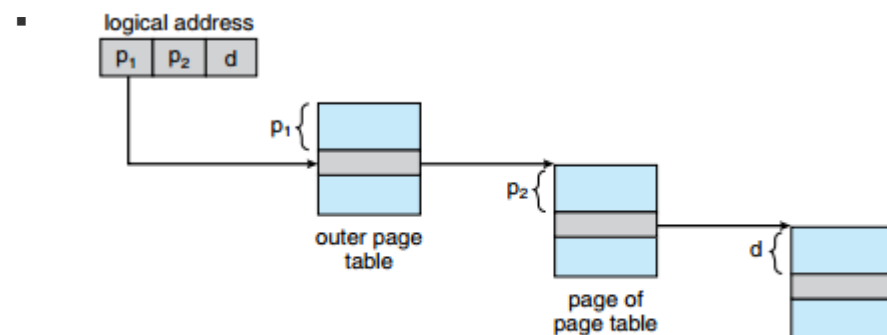
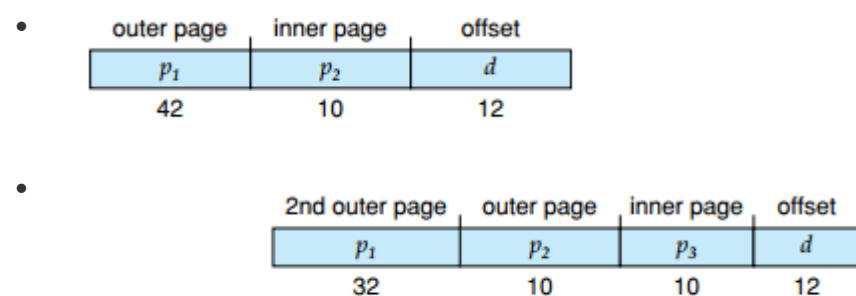


Figure 8.18 Address translation for a two-level 32-bit paging architecture.

- Most modern computer systems support a large logical address space (2^{32} to 2^{64}). In such an environment, the page table itself becomes excessively large. For example, consider a system with a 32-bit logical address space. If the page size in such a system is 4 KB (2^{12}), then a page table may consist of up to 1 million entries ($2^{32}/2^{12}$). Assuming that each entry consists of 4 bytes, each process may need up to 4 MB of physical address space for the page table alone. Clearly, we would not want to allocate the page table contiguously in main memory. One simple solution to this problem is to divide the page table into smaller pieces. We can accomplish this division in several ways.

One way is to use a two-level paging algorithm, in which the page table itself is also paged (Figure 8.17). For example, consider again the system with

- 64 bit logical address space



The outer page table is still 2^{34} bytes (16 GB) in size.

- 64 bit UltraSPARC would required seven levels of paging - a prohibitive number of memory accesses- to translate each logical address spcae,
- Thus hierarchical paging is inappropriate for 64 bit systems.

• Hashed Page Tables

- A common approach for handling address spaces larger than 32 bits is to use a hashed page table, with the hash value being the virtual page number.
- Each element consists of three fields:
 1. The virtual page number
 2. The value of the mapped page frame
 3. A pointer to the next element in the linked list
- The algorithm works as follows:
 1. The virtual page number in the virtual address is hashed into the hash table.
 2. The virtual page number is compared with field 1 in the first element in the linked list.
 3. If there is a match, the corresponding page frame (field 2) is used to form the desired physical address.
 4. If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.

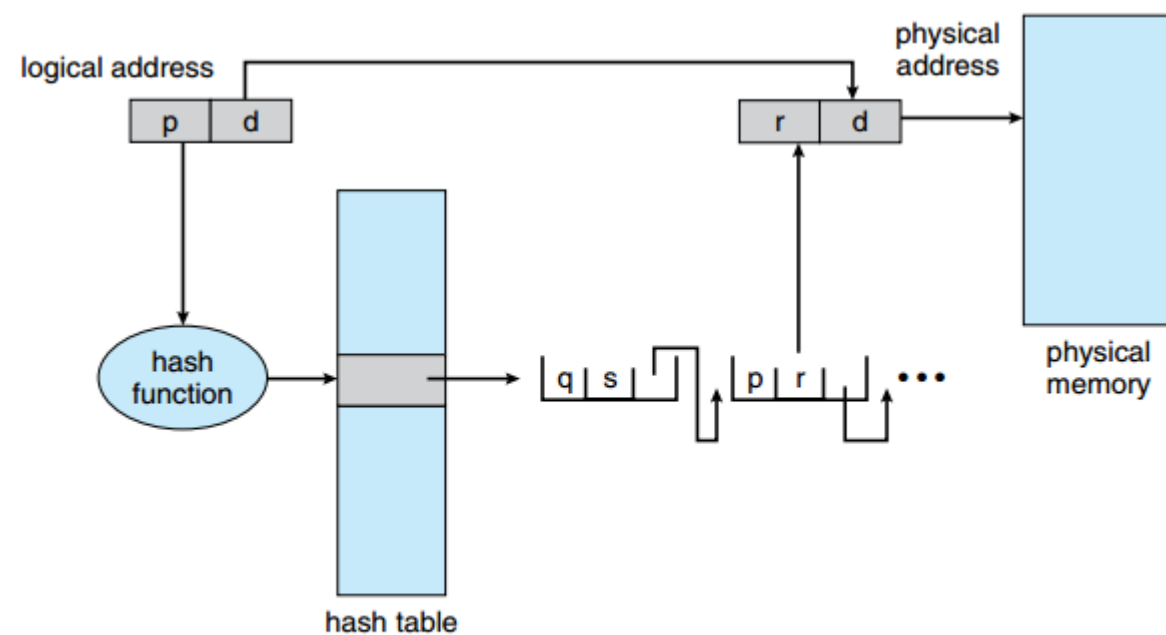


Figure 8.19 Hashed page table.

- **Clustered Page Tables**
 - A variation of this scheme
 - Except that each entry in the hash table refers to several pages rather than a single page.
 - Useful for sparse address space, where memory references are contiguous and scattered throughout the address space.
- **Inverted Page tables**
 - Because each process with one entry for each page would require a lot of space, given a virtual address of the page, each memory translation would require the OS to calculate the physical memory location from the reference address. And also these entries could be in Millions, can not be stored even by TLBs
 - An **inverted page table** has one entry for each real page, OBB
 - Each entry has virtual address of the page OBB
 - The advantage point is, it has info about the process that owns the page
 - Thus, only one page is in the system which has everything, no need to have separate page tables for each process

- Thus, **only one page is in the system, which has everything**, no need to have separate page tables for each process

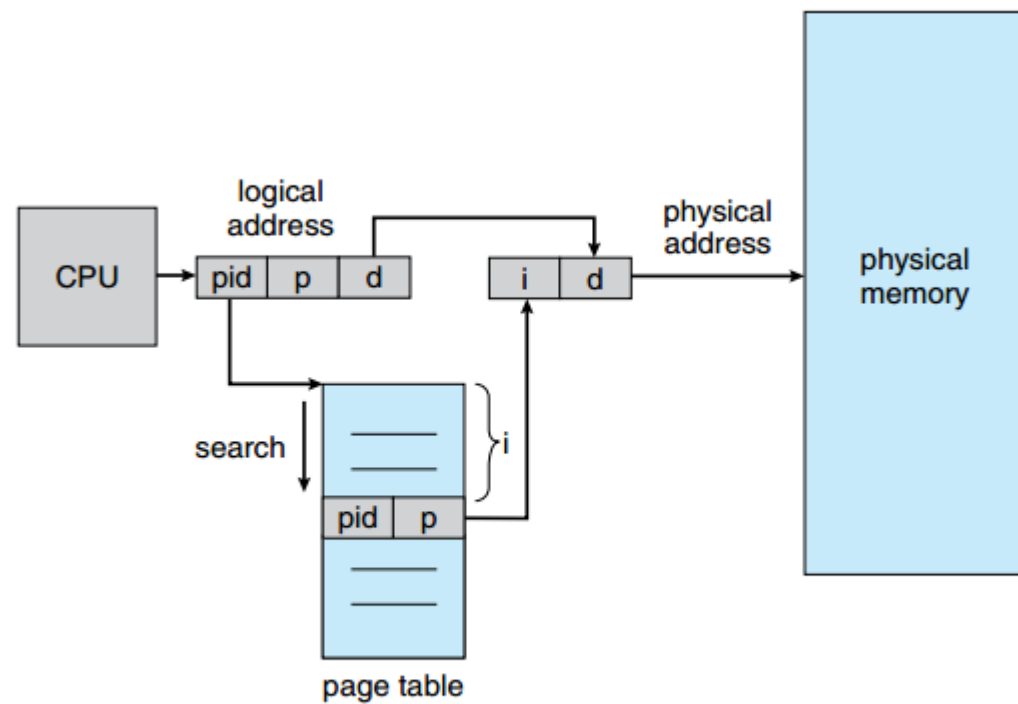


Figure 8.20 Inverted page table.

<process-id, page-number, offset>.

- Each inverted page-table entry is a pair **<process-id, page-number>** where the **process-id assumes the role of the address-space identifier**.
 - When a memory reference occurs, part of the virtual address, consisting of **<process-id, pagenumber>**, is presented to the memory subsystem.
 - The inverted page table is then searched for a match. If a match is found—say, at entry **i**—then the physical address **<i, offset>** is generated.
 - If no match is found, then an illegal address access has been attempted.
- ADVANTAGES:**
 - Decreases memory usage for each process (no page stored)
- DISADVANTAGES:**
 - Because the inverted page table is sorted by physical address, but lookups occur on virtual addresses, the **whole table might need to be searched before a match is found**.
 - To **alleviate this problem, we use a hash table**, to limit the search to one—or at most a few—page-table entries.
 - Of course, **each access to the hash table adds a memory reference to the procedure**, so one virtual memory reference requires at least two real memory reads—one for the **hash-table entry** and **one for the page table**.
 - (Recall that the TLB is searched first, before the hash table is consulted, offering some performance improvement.)
 - Systems that use **inverted page tables have difficulty implementing shared memory**.
 - Only one **PID can be stored in ASID or page table entry**, because there is only one virtual page entry for every physical page
- Oracle SPARC Solaris**
 - Two hash tables:
 - kernel processes
 - user processes
 - Each hash table entry represent a contiguous area of mapped virtual memory, **Clustered Hash rable approach**

- So the CPU implements TLB that holds **TTEs (Translation table Entries)**
 - Match found then okay,
 - No match found, then **the cache called TSB stores these TTEs**, which include an **entry per recently accesses page**.
 - If no match in TSB also, then the kernel is interrupted for searching the hash table.
 - The kernel creates a TTE from the hash table and stores it in teh TSB, finally the control returns to the MMU, which completes the address translation and retrieves the requested byte or word from main memory.

Open in desktop app?

☐ Remember my choice

No, thank you

Open in app